

2026년 오픈소스 개발자대회 결과보고서

팀 명	퍼즐	팀 인원(팀장 포함)	2명 (이정우, 이서인)
참가부문	일반	과제유형	자유과제

□ 결과보고서

프로젝트 개요	
프로젝트명	Purchase Research Agent (근거 기반 구매 조사 에이전트)
프로젝트 등록 URL	https://github.com/leeseoin/opensource-competition
시연 영상	미기재 이번 초안에서는 제외했습니다. 결과보고서 양식은 3분 이내 시연 영상 제작 → 유튜브 업로드 → 링크 기재를 필수(必)로 안내하므로 최종 제출 전 팀에서 다시 확인해 주세요.
프로젝트 소개	자연어로 말한 구매 조건을 실제 쇼핑몰 공개 상품·리뷰 데이터로 근거를 붙여 비교해 주고, 선택한 상품은 구매 직전 가격·재고를 다시 확인해 주는 구매 조사 에이전트
프로젝트 세부 내용	
개발 배경 및 목적	여러 쇼핑몰을 오가며 가격·재고·리뷰를 직접 비교하는 일은 시간이 오래 걸리고, 가격이 계속 바뀌기 때문에 한 번 모은 정보로는 부족합니다. 또한 챗봇형 서비스가 흔히 겪는 문제는 AI가 그럴듯하지만 근거 없는 답을 내놓는 것입니다. 이 프로젝트는 두 가지를 목표로 삼았습니다. - 쇼핑몰 화면을 정기적으로 자동 수집해 상품명·가격·리뷰·옵션을 최신 상태로 유지하고, 수집한 값이 실제 화면과 다르지 않은지 스스로 검증하는 것 - 사용자가 자연어로 구매 조건을 말하면 부족한 조건을 되묻고, DB에 쌓인 출처가 있는 공개 데이터만 근거로 후보를 제시해 "출처를 알 수 없는 답"이 나오지 않도록 하는 것 이를 위해 팀은 서로 다른 언어(Go, Python, Java, TypeScript)로 역할을 나눠 개발했고, 언어가 달라도 안전하게 데이터를 주고받을 수 있도록 <code>contracts</code> 폴더에 JSON Schema로 서비스 간 계약을 먼저 정한 뒤 각 구성요소를 구현했습니다.

개발 환경	• 언어/런타임: Python 3.12(수집기), Go 1.25(수집기 비교/복구 기준), Java 21 · Spring Boot 4.1.0(백엔드), TypeScript/Node.js · Next.js 16(웹), TypeScript(MCP Server) • 데이터 저장소: PostgreSQL 16 + pgvector 0.8.2(상품/근거/벡터 검색), Redis 7.2(속도 제한·중복 방지), RabbitMQ 4.2(수집 작업/결과 큐) — <code>compose.yaml</code> 로 로컬 실행 • 쇼핑몰 자동 수집: Crawl4AI(Playwright 기반 브라우저 자동화) · BeautifulSoup4(Python), 자체 HTTP 클라이언트(Go) • AI 연동: Model Context Protocol(MCP) TypeScript SDK로 Codex/Claude Code와 백엔드 REST API를 연결, Next.js 서버가 Codex CLI를 읽기 전용으로 호출해 사용자 발화를 구매 조건 JSON으로 구조화 • 패키지/버전 고정: <code>uv(pyproject.toml / uv.lock)</code>, Gradle, <code>npm(package-lock.json)</code> • 실행/검증 자동화: 저장소 루트 Makefile 의 <code>make env</code>, <code>make infra-up</code>, <code>make collector-run</code>, <code>make collector-worker</code>, <code>make product-backend-run</code>, <code>make web-dev</code>, <code>make test</code>, <code>make check</code>, <code>make docs-check</code> • 테스트: pytest(Python), <code>go test</code> (Go), JUnit + Testcontainers(Java, PostgreSQL/RabbitMQ 통합 테스트), Node.js <code>node:test</code> (Next.js)
시스템 구성 및 아키텍처	사용자 ↓ Next.js 구매 채팅 화면 (frontend/purchase-web) ↓ server route Codex/Claude Code Gateway - 발화를 구매 조건 JSON으로 구조화 ↓ MCP MCP Server (services/mcp-server) - AI 도구 ↔ REST API 변환, DB/RabbitMQ 직접 접근 없음 ↓ REST API Spring Boot Product Backend (services/product-backend) ├── PostgreSQL(+pgvector): 상품, 가격/재고 snapshot, 근거, 작업 상태 저장 ├── RabbitMQ: 수집 작업 발행 · 결과 소비 └── Redis: 속도 제한 · 중복 방지 · 진행 상태 ↓ Collector Worker (services/python-collector 主, services/collector Go 비교 기준) ↓ ABC마트 · 29CM 공개 상품/리뷰 페이지 • 왜 MCP Server와 Product Backend를 나누는가: 웹 화면 등 일반 기능은 Backend REST API를 직접 쓰고, AI 도구 호출만 MCP Server를 거치게 해 MCP 규격이나 AI 실행 환경이 바뀌어도 상품 저장·수집 로직을 함께 바꾸지 않도록 분리했습니다. • 핵심 데이터 흐름: 사용자 질문마다 쇼핑몰을 다시 확인하지 않고, 백그라운드로 쌓아둔 PostgreSQL 데이터를 먼저 조회합니다. 근거가 부족할 때만 제한된 추가 수집을 요청하고, 사용자가 상품을 선택하면 가격·재고·옵션을 다시 한 번 재검증합니다. • 계약(contracts/): 큐 메시지(<code>collection/v1</code>), 검색 요청/결과(<code>collector/v1</code>), 실제 ABC마트 수집값과 백엔드 저장값을 대조한 차이점 문서(<code>collector/v1-abcmart</code>), Python/Go 수집기 결과를 비교하기 위한 통합 형식(<code>collector/unified</code>)을 JSON Schema로 관리합니다.

프로젝트 주요 기능

프로젝트 상세 내용

핵심 기능 및 특징

- **자연어 구매 조건 파악:** "면접용 구두 찾아줘" 같은 발화에서 부족한 조건(사이즈, 예산 등)을 되묻고, 확정된 조건을 필수/선택으로 나눠 검색에 사용합니다.
- **근거 기반 하이브리드 상품 검색:** PostgreSQL 전문 검색(FTS)·trigram 유사도를 기본으로 하고, 선택적으로 pgvector 기반 임베딩 검색을 결합해 후보를 재정렬합니다. 추천 결과에는 항상 공개 출처(sourceUrl)와 일치/불일치/확인 불가 근거가 함께 표시됩니다.
- **백그라운드 큐 기반 수집:** 검색 요청은 RabbitMQ 작업으로 발행되고 Collector Worker가 순서대로 처리한 뒤 결과를 다시 큐로 돌려줍니다. 한 번에 요청이 몰리거나 쇼핑몰 응답이 느려져도 전체 서비스가 멈추지 않는 구조입니다.
- **수집 결과 자체 검증:** 검색 API 응답과 실제 상세 화면을 서로 대조해 값이 다르면 "다르다"는 사실 자체를 기록으로 남깁니다.
- **세분화된 실패 상태와 재시도 정책:** 수집 상태를 `success` / `partial` / `blocked` / `unsupported` / `temporarily_unavailable` 로 구분하고, 일시적 오류(429, 5xx)만 재시도하며 로그인·차단(401/403)은 즉시 실패 처리해 불필요한 재시도를 막습니다.
- **안전한 수집 원칙:** `robots.txt` 허용 범위 준수, 인증서 검증 우회 금지, 자동 리다이렉트 금지를 정적 검사 스크립트로 강제하고, 리뷰 작성자 식별정보와 리뷰 이미지 파일은 아예 수집하지 않습니다.
- **Python/Go 이중 구현 비교:** 같은 계약을 두 언어로 구현해 최대 1만 건 규모의 수집 성능과 정확도를 비교했고, 현재 운영 전환 기준은 Python 구현입니다.
- **AI 실행 환경과 저장소 책임 분리:** MCP Server는 PostgreSQL·RabbitMQ·판매처에 직접 접근하지 않고 오직 Backend REST API만 호출하도록 경계를 강제합니다.

개발 과정(주요 단계)

1. 한 쇼핑몰(ABC마트)의 목록 페이지에서 상품명·가격을 가져오는 최소 구현으로 시작.
2. 수집값이 실제 화면과 다를 수 있음을 발견해 검색 결과-상세 화면 대조 검증 단계 추가.
3. 대량 수집을 안정적으로 처리하기 위해 즉시 처리 방식을 RabbitMQ 큐 기반 처리로 전환.
4. Java 백엔드와 Python/Go 수집기가 병렬 개발되며 데이터 형식이 어긋나는 문제가 반복돼, JSON Schema로 계약을 먼저 정하고 실제 코드가 이를 지키는지 검사하는 절차를 추가.
5. 두 번째 쇼핑몰(29CM)을 추가하며 상세 화면 수집 로직을 확장.
6. Spring Boot Backend에 Flyway/JPA 기반 상품 저장, 수집 작업 발행/결과 소비, pgvector 벡터 검색을 순서대로 구현.
7. MCP Server와 Next.js Agent Gateway를 붙여 Codex/Claude Code가 조건 확인 → DB 우선 검색 → 부족 시 제한된 추가 수집 → 재검증까지 이어지는 상태 기반 흐름 (Agent Run)을 구현.

구동 및 시연

1. 저장소 루트에서 `.env` 준비 후 `make infra-up` 으로 PostgreSQL/Redis/RabbitMQ를 실행합니다.
2. 각각 별도 터미널에서 `make product-backend-run`, `make collector-worker` (또는 `make collector-run`), `make web-dev` 를 실행합니다.

	3. Product Backend의 Swagger UI(http://localhost:8080/swagger-ui.html)에서 수집 작업 등록 → 진행 상태 조회 → 저장된 상품·가격·근거 조회까지 확인할 수 있습니다. 4. Next.js 화면에서 자연어로 구매 조건을 입력하면 조건 확인 → 후보 비교 → 재검증까지 한 흐름으로 진행되는 것을 확인할 수 있습니다. 5. 기본 검증은 <code>make test</code>, <code>make check</code> 로 실행합니다.
--	---

기대효과 및 활용 분야	
--------------	--

향후 확장성 및 기대효과	현재는 ABC마트·29CM 두 곳, 신발·의류 카테고리만 지원하지만 판매처별 Collector Adapter와 계약(JSON Schema)만 추가하면 새 쇼핑몰·상품군으로 확장할 수 있는 구조입니다. 서비스 간 데이터 형식을 계약으로 먼저 정하고 실제 코드가 이를 지키는지 검사하는 방식은 이번 프로젝트뿐 아니라 여러 언어·여러 팀원이 동시에 개발하는 다른 프로젝트에도 그대로 적용할 수 있습니다. 근거를 투명하게 남기는 검색·재검증 구조는 쇼핑 비교를 넘어 "AI 답변에 출처가 필요한" 다른 도메인(여행, 가전 등)에도 확장할 수 있습니다.
---------------	--

기타	
----	--

프로젝트의 혁신성 및 차별성	• Go/Python/Java/TypeScript로 언어가 서로 다른 4개 구성요소를 JSON Schema 계약으로 안전하게 연결하고, 실제 수집값과 백엔드 저장값을 하나씩 대조해 어긋나는 지점을 문서로 남긴 점. • AI(Codex/Claude Code)는 자연어 조건을 구조화하는 역할만 맡고, 실제 상품 추천의 근거는 PostgreSQL에 저장된 공개 출처 데이터로만 만들어 "AI가 지어낸 답"이 나오지 않도록 설계 경계를 강제한 점(MCP Server는 DB/큐/판매처에 직접 접근하지 않음). • 같은 계약으로 Python/Go 수집기를 이중 구현해 실제 수집 성능과 정확도를 비교 검증한 점.
한계점 및 향후 발전 로드맵	• 지원 쇼핑몰이 두 곳뿐이라 확장이 필요합니다. • 수집 작업의 영구 상태 관리(BACKEND-002), 리뷰 신호 추출·상품 비교(ANALYSIS-001), 검토형 구매 도메인 Wiki 고도화(ANALYSIS-003) 등 계획/부분 구현 상태인 기능이 남아 있습니다(docs/planning/Purchase_Research_Agent_기능_목록.md 참고). • 저장소 루트에 LICENSE 파일이 아직 없어 제출 전 팀 협의로 OSI 승인 라이선스(MIT, Apache-2.0 등)를 선택해 추가해야 합니다. • Agent Gateway가 상용 API(Codex CLI)에 의존해 구매 조건을 구조화하는 부분이 대회 운영규정 제9조의 어느 예외에 해당하는지 팀 자체 판단으로는 확정하지 못해 운영사무국 서면 확인이 필요합니다(붙임2 참고).
소감 및 후기	확인 필요 각 팀원이 직접 겪은 어려움과 협업 과정을 1인칭으로 작성해 주세요. (예: 서로 다른 언어로 개발하며 겪은 계약 불일치, AI 도구 활용 범위를 정하며 논의한 과정 등)

붙임1

SBOM(소프트웨어 자재명세서)

번호	라이브러리명	버전	라이선스	공식 저장소 URL	사용 목적 및 주요 기능
1	Model Context Protocol TypeScript SDK	1.30.0	MIT	github.com/modelcontextprotocol/typescript-sdk	AI 도구 요청을 Backend REST API로 연결(MCP Server 핵심) / 라이브러리로 불러 씀
2	RabbitMQ AMQP 0-9-1 Go Client	1.10.0	BSD-3-Clause	github.com/rabbitmq/amqp091-go	Go Collector ↔ Backend 수집 작업/결과 메시지 처리 / 라이브러리로 불러 씀
3	pgvector	0.8.2	PostgreSQL License	github.com/pgvector/pgvector	PostgreSQL 벡터 검색 (상품 후보 재정렬) / DB 확장으로 설치
4	Crawl4AI	0.9.2	Apache-2.0	github.com/unclecode/crawl4ai	Python 수집기의 쇼핑몰 브라우저 자동 방문/수집 / 라이브러리로 불러 씀
5	Spring Boot	4.1.0	Apache-2.0	github.com/spring-projects/spring-boot	Product Backend 웹/JPA/AMQP 프레임워크 / 라이브러리로 불러 씀
6	Next.js	16.2.12	MIT	github.com/vercel/next.js	사용자 구매 채팅 화면과 서버 라우트 프레임워크 / 라이브러리로 불러 씀
7	React	19.2.4	MIT	github.com/facebook/react	화면 UI 컴포넌트 렌더링 / 라이브러리로 불러 씀
8	FastAPI	0.141.1	MIT	github.com/fastapi/fastapi	Python 수집기 HTTP API 서버 / 라이브러리로 불러 씀
9	Flyway	12.4.0	Apache-2.0	github.com/flyway/flyway	PostgreSQL 스키마 마이그레이션 / 라이브러리로 불러 씀
10	Playwright(Python)	1.62.0	Apache-2.0	github.com/microsoft/playwright-python	Crawl4AI가 사용하는 브라우저 자동 조작 엔진 / 라이브러리로 불러 씀

THIRD_PARTY_NOTICES.md 에 기록된 각 서비스(Go Collector, Python Collector, Spring Boot Product Backend, Next.js Web, MCP Server)의 직접 의존성 중 붙임1 가이드의 우선순위(① GPL·AGPL·LGPL → ② 핵심 기능 → ③ 주요 프레임워크·SDK → ④ 빌드·실행 도구)에 따라 팀 전체 기준 10개를 선별했습니다. GPL·AGPL·LGPL 계열 라이선스를 쓰는 라이브러리는 없습니다. 버전은 각 서비스의 lock 파일(`uv.lock`, `go.sum`, `package-lock.json`, Gradle 의존성) 기준 실제 설치 버전이며, 제출 전 각 서비스에서 `./gradlew dependencies`, `go list -m all`, `uv pip list`, `npm ls --depth=0` 로 최신 상태를 한 번 더 확인하는 것을 권장합니다.

붙임2

AI 모델 활용 및 라이선스 기술 명세서

확인 필요 이 프로젝트는 실행 중 상품 추천의 근거를 만드는 로직(PostgreSQL 전문/벡터 검색)에는 AI 모델을 사용하지 않습니다. 다만 Next.js Agent Gateway가 사용자 발화를 구매 조건 JSON으로 구조화할 때 Codex CLI(상용 API)를 서버에서 호출하고, MCP Server는 AI 모델과의 연동 통로 자체를 만드는 커넥터입니다. [부록2] 2장의 "MCP 에이전트 프레임워크 연동 도구는 예외적으로 허용"에 해당할 가능성이 높다고 판단했으나, 팀 자체 체크리스트(docs/planning/오픈소스_개발자대회_규정_대응_체크리스트.md)에서도 최종 판단은 운영사무국 서면 확인 후로 미뤄 둔 상태입니다. **제출 전 반드시 운영사무국에 문의해 붙임2를 그대로 돌지, 삭제하지, 유형 체크가 필요한지 확인해 주세요.**

1. AI 모델 활용 유형	
확인 필요	현재 판단으로는 유형 1·2·3 어디에도 해당하지 않으며, 자체 학습·파인튜닝한 가중치는 없습니다.
2. 기반(베이스) 모델 정보	
해당 없음(자체 학습·파인튜닝 없음)	
3. 데이터셋 정보 및 가중치 배포 명세	
해당 없음(자체 학습·파인튜닝 없음)	
4. 소스코드 라이선스 및 개발 환경 정보	
직접 작성한 코드의 오픈 소스 라이선스	확인 필요 저장소에 LICENSE 가 아직 없어 팀 협의 후 OSI 승인 라이선스(예: MIT, Apache-2.0)를 선택해야 합니다.
학습/추론 소스코드 공개 저장소 URL	https://github.com/leeseoin/opensource-competition
상용 AI 보조도구 활용 여부 및 범위	개발 전반에 걸쳐 OpenAI Codex와 Anthropic Claude Code를 저장소 분석·설계 대화·Go/Java/TypeScript/Python 코드 초안 작성·테스트 보조·문서 작성에 활용했습니다. 모든 변경은 팀원이 git diff 와 관련 단위/통합 테스트로 검토했습니다. 실행 시에는 Next.js Agent Gateway가 Codex CLI를 읽기 전용/비대화형으로 호출해 사용자 발화를 구매 조건 JSON으로 구조화하는 데에만 사용합니다. 상세 사용 이력은 저장소 AI_USAGE.md 에 공개돼 있습니다.

제출 전 팀 확인 체크리스트

- 시연영상: 이번 초안에서 제외했으나, 결과보고서 양식은 3분 이내 시연 영상 제작 → 유튜브 업로드 → 링크 기재를 필수(必)로 안내하므로 최종 제출 전 팀 확인 필요
- 저장소 루트에 LICENSE 추가(OSI 승인 라이선스, 팀 협의 필요)
- MCP/Agent의 Codex CLI 런타임 호출이 불임2 대상인지 운영사무국 서면 확인
- 소감 및 후기 항목을 팀원이 1인칭으로 작성
- SBOM 버전을 제출 직전 각 서비스 lock 파일 기준으로 재확인